

RAPPORT DE PROJET

Système Académique de Suivi et Gestion des Notes

Responsable : Adnane SOUHA

Crée par : Amine MOUADRI

Module : Programation Python 1

Développée avec Python et Tkinter

Année Académique 2025-2026

TABLE DES MATIÈRES

- 1. Introduction 3
- 2. Problématique..... 4
- 3. Objectifs 5
- 4. Cahier des Charges..... 6
 - 4.1 Besoins Fonctionnels 6
 - 4.2 Besoins Non Fonctionnels..... 7
- 5. Choix Techniques 8
- 6. Architecture du Système 9
- 7. Conception 10
- 8. Fonctionnalités Principales 12
- 9. Réalisation..... 14
- 10. Tests et Validation..... 15
- 11. Difficultés et Solutions 16
- 12. Limites et Perspectives..... 17
- 13. Conclusion 18

1. Introduction

Dans le contexte actuel de la digitalisation des processus administratifs au sein des établissements d'enseignement, la gestion efficace des notes étudiants représente un enjeu majeur pour les institutions académiques. Le présent projet s'inscrit dans cette démarche de modernisation en proposant une solution logicielle complète dédiée au suivi et à la gestion des résultats académiques.

Le Système Académique de Suivi et Gestion des Notes (SASGN) est une application desktop développée en Python qui permet aux enseignants et aux administrateurs de gérer de manière efficace les notes des étudiants. Cette application offre une interface graphique intuitive construite avec la bibliothèque Tkinter, facilitant ainsi l'interaction avec les données académiques.

2. Problématique

La gestion traditionnelle des notes académiques présente plusieurs défis majeurs pour les établissements d'enseignement :

1. La manipulation manuelle des données sur papier ou sur des feuilles Excel dispersées engendre des erreurs et une perte de temps considérable.
2. Le calcul manuel des moyennes et l'attribution des mentions sont sujets à des erreurs humaines.
3. La génération des relevés de notes officiels demande beaucoup de temps et de ressources.
4. La difficulté à suivre l'évolution des performances des étudiants au fil du temps.

Ces problèmes soulignent la nécessité d'une solution informatique intégrée qui automatise les processus de gestion des notes tout en garantissant la fiabilité et la traçabilité des données.

3. Objectifs

Le projet vise à atteindre les objectifs suivants :

Objectifs Généraux

5. Développer une application desktop complète pour la gestion des notes étudiants.
6. Automatiser le calcul des moyennes et la détermination des validations.
7. Faciliter la génération des relevés de notes au format PDF.

Objectifs Spécifiques

8. Permettre l'importation des données depuis des fichiers Excel existants.
9. Offrir une interface pour la saisie manuelle des données.
10. Implémenter les opérations CRUD (Create, Read, Update, Delete) pour les étudiants et les modules.
11. Générer des statistiques de classe (moyenne générale, notes min/max).
12. Produire des relevés de notes personnalisés avec mention et validation.

4. Cahier des Charges

4.1 Besoins Fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités que le système doit offrir :

Fonctionnalité	Description
Import Excel	Importer les données depuis un fichier .xlsx
Saisie manuelle	Saisir les données sans fichier externe
Ajout étudiant	Ajouter un nouvel étudiant avec ses notes
Suppression	Supprimer un étudiant ou un module
Modification notes	Modifier les notes d'un étudiant
Statistiques	Afficher les stats de classe (min, max, moyenne)
Relevé PDF	Générer un relevé de notes officiel
Export Excel	Exporter les données vers un fichier Excel

4.2 Besoins Non Fonctionnels

Les besoins non fonctionnels définissent les contraintes et qualités du système :

Critère	Description	Niveau
Performance	Temps de réponse < 2s	Élevé
Fiabilité	Calculs exacts des moyennes	Élevé
Ergonomie	Interface intuitive et claire	Moyen
Portabilité	Fonctionne sur Windows/Linux	Moyen
Maintenabilité	Code modulaire et documenté	Élevé

5. Choix Techniques

Le développement du système s'appuie sur les technologies et outils suivants :

Langage de Programmation

Python a été choisi comme langage principal pour sa simplicité syntaxique, sa richesse en bibliothèques et sa grande communauté de développeurs. Sa versatilité permet de développer rapidement des applications desktop performantes.

Interface Graphique

Tkinter, la bibliothèque GUI standard de Python, a été retenue pour la création de l'interface utilisateur. Son intégration native avec Python et sa légèreté en font un choix idéal pour des applications desktop simples et efficaces.

Manipulation de Données

La bibliothèque Pandas est utilisée pour la lecture et l'écriture des fichiers Excel, offrant des fonctionnalités puissantes pour la manipulation de données tabulaires.

Génération de PDF

ReportLab est employé pour la génération des relevés de notes au format PDF, permettant une personnalisation complète du document final.

Technologie	Usage	Version
Python	Langage principal	3.12.9
Tkinter	Interface graphique	Standard
Pandas	Manipulation Excel	1.x
ReportLab	Génération PDF	3.x

6. Architecture du Système

L'architecture du système suit une organisation modulaire qui sépare les responsabilités et facilite la maintenance :

Organisation des Fichiers

Fichier	Description
main.py	Point d'entrée de l'application
data_loader.py	Chargement depuis un fichier Excel ou initialisation manuelle
Gestion.py	Panneau de gestion principal
FONCTIONS.py	Fonctions utilitaires
releve_note.py	Génération des relevés PDF

Description des Modules

Le module `main.py` constitue le point d'entrée de l'application. Il affiche l'écran d'initialisation permettant à l'utilisateur de choisir entre l'importation d'un fichier Excel ou la saisie manuelle des données.

Le module `data_loader.py` gère deux fonctions principales : l'une permet de charger et lire les données depuis un fichier Excel, tandis que l'autre permet d'initialiser les données par saisie manuelle via une interface utilisateur. Il assure ainsi la préparation des données avant leur traitement dans le système.

Le module `Gestion.py` représente le cœur de l'application avec le panneau de gestion principal. Il contient toutes les fonctionnalités de gestion des étudiants et des notes.

Le module `FONCTIONS.py` regroupe les fonctions utilitaires communes, notamment le calcul des moyennes, la détermination des validations, et le style des boutons.

Le module `releve_note.py` est dédié à la génération des relevés de notes au format PDF avec le logo de l'établissement.

7. Conception

7.1 Structure des Données

Les données sont organisées sous forme d'une liste de dictionnaires, où chaque dictionnaire représente un étudiant avec ses informations et ses notes :

```
data_dict = [ { 'ID': 'E001', 'NOM': 'Etudiant A',  
'Module 1': 15.5, 'Module 2': 14.0, 'moyenne': 14.75,  
'Validation': 'Validé' }, ... ]
```

7.2 Logique de Traitement

Le système implémente plusieurs algorithmes de traitement :

Calcul de la Moyenne

La moyenne d'un étudiant est calculée en faisant la somme de toutes ses notes de modules divisée par le nombre de modules. Les champs ID, NOM, moyenne et Validation sont exclus du calcul.

Détermination de la Validation

Un étudiant est considéré comme validé si sa moyenne générale est supérieure ou égale à 10/20.

Attribution des Mentions

Moyenne	Mention
18 - 20	Très Bien avec félicitations
16 - 17.99	Très Bien
13 - 15.99	Bien
11 - 12.99	Assez Bien
8 - 10.99	Insuffisant

Moyenne	Mention
0 - 7.99	Très insuffisant

8. Fonctionnalités Principales

8.1 Importation Excel

L'application permet d'importer des données depuis un fichier Excel (.xlsx) contenant les colonnes ID, NOM et les notes des différents modules. Une validation vérifie la présence de la colonne NOM obligatoire.

8.2 Saisie Manuelle

En l'absence de fichier Excel, l'utilisateur peut saisir manuellement les données des étudiants et de leurs notes module par module.

8.3 Gestion des Étudiants

- 13. Ajout d'un nouvel étudiant avec toutes ses informations.
- 14. Suppression d'un étudiant existant.
- 15. Recherche d'un étudiant par ID ou nom.
- 16. Affichage des détails et statistiques individuelles.

8.4 Gestion des Modules

- 17. Ajout d'un nouveau module pour tous les étudiants.
- 18. Suppression d'un module existant.
- 19. Modification des notes par module.

8.5 Statistiques

Le panneau de gestion affiche en temps réel :

- 20. La note maximale de la classe avec le nom de l'étudiant.
- 21. La note minimale de la classe avec le nom de l'étudiant.

22. La moyenne générale de la classe.

23. Le nombre d'étudiants et de modules.

8.6 Génération de Relevés

L'application génère des relevés de notes PDF officiels incluant :

24. Le logo et l'en-tête de l'établissement (Université Moulay Ismaïl).

25. Les informations de l'étudiant (ID, Nom, Filière).

26. Le tableau des notes par module.

27. La moyenne, la mention et la validation.

9. Réalisation

Le développement du projet a suivi une approche itérative en plusieurs phases :

Phase 1 : Analyse et Conception

Cette phase a consisté à analyser les besoins, définir les fonctionnalités et concevoir l'architecture du système. Les choix techniques ont été validés et la structure des données a été définie.

Phase 2 : Développement des Modules de Base

Le module de lecture Excel (`data_loader.py`) et les fonctions utilitaires (`FONCTIONS.py`) ont été développés en premier pour constituer la fondation du système.

Phase 3 : Développement du Panneau de Gestion

Le module `Gestion.py` a été développé pour intégrer toutes les fonctionnalités de gestion avec une interface utilisateur complète incluant les menus, boutons et tableaux.

Phase 4 : Génération des Relevés

Le module `releve_note.py` a été implémenté pour produire des documents PDF professionnels avec le logo de l'établissement.

Phase 5 : Tests et Intégration

Des tests ont été effectués sur chaque module individuellement, puis une phase d'intégration a permis de valider le fonctionnement global du système.

10. Tests et Validation

La validation du système a été réalisée à travers plusieurs types de tests :

Tests Unitaires

- 28. Vérification du calcul correct des moyennes.
- 29. Test de la détection de la colonne NOM dans les fichiers Excel.
- 30. Validation de l'attribution des mentions selon les seuils définis.

Tests d'Intégration

- 31. Importation complète d'un fichier Excel et affichage des données.
- 32. Ajout d'un étudiant et mise à jour automatique des statistiques.
- 33. Génération d'un relevé de notes et vérification du contenu PDF.

Tests de Validation

- 34. Test avec un fichier Excel vide pour vérifier le message d'erreur.
- 35. Test avec un fichier sans colonne NOM pour valider la validation.

Test	Résultat	Statut
Import fichier Excel valide	Données chargées correctement	Réussi
Import fichier sans colonne NOM	Message d'erreur affiché	Réussi
Calcul moyenne étudiant	Résultat correct	Réussi
Ajout nouvel étudiant	Étudiant ajouté et stats mises à jour	Réussi
Génération relevé PDF	PDF crée avec bon contenu	Réussi

Test	Résultat	Statut
Export vers Excel	Fichier créé et lisible	Réussi

11. Difficultés Rencontrées et Solutions

Difficulté 1 : Gestion des Scrollbars

Problème : L'ajout d'un grand nombre d'étudiants rendait l'interface difficile à utiliser.

Solution : Implémentation d'une scrollbar verticale dans le module d'ajout de modules en utilisant Canvas et Scrollbar de Tkinter.

Difficulté 2 : Mise à Jour Dynamique du Tableau

Problème : Le tableau Treeview ne se mettait pas à jour automatiquement après une modification.

Solution : Création d'une fonction `update_stats()` qui rafraîchit toutes les données affichées après chaque opération.

Difficulté 3 : Gestion des Clés de Dictionnaire

Problème : La première clé du dictionnaire (ID) pouvait varier selon les fichiers Excel.

Solution : Utilisation de `next(iter(etudiant))` pour récupérer dynamiquement la première clé de chaque étudiant.

Difficulté 4 : Génération de PDF

Problème : Intégration du logo et mise en forme du relevé de notes.

Solution : Utilisation de ReportLab avec des tableaux stylisés et des paragraphes formatés.

12. Limites et Perspectives

12.1 Limites du Projet

- 36. L'application est une application desktop locale sans base de données centralisée.
- 37. Pas de système d'authentification des utilisateurs.
- 38. Pas de sauvegarde automatique des données.
- 39. L'interface graphique pourrait être modernisée avec des frameworks plus récents.

12.2 Perspectives d'Amélioration

- 40. Migration vers une architecture web avec une base de données centralisée.
- 41. Implémentation d'un système d'authentification multi-utilisateurs.
- 42. Ajout de graphiques statistiques (courbes, histogrammes).
- 43. Export des données vers d'autres formats (CSV, JSON).
- 44. Développement d'une version mobile pour les étudiants.
- 45. Intégration avec les systèmes d'information existants des universités.

13. Conclusion

Le Système Académique de Suivi et Gestion des Notes représente une solution complète et fonctionnelle pour la gestion des résultats académiques. Ce projet a permis de mettre en pratique les concepts fondamentaux du développement d'applications desktop avec Python et de démontrer l'efficacité des bibliothèques disponibles dans l'écosystème Python.

Les fonctionnalités implémentées couvrent l'ensemble du cycle de vie de la gestion des notes, depuis l'importation des données jusqu'à la génération des relevés officiels. L'interface utilisateur intuitive facilite l'adoption du système par les enseignants et administrateurs.

Les résultats obtenus sont conformes aux objectifs initiaux :

1. Automatisation complète du calcul des moyennes et des validations.
2. Génération rapide et professionnelle des relevés de notes.
3. Interface de gestion centralisée et efficace.
4. Flexibilité d'utilisation avec ou sans fichier Excel.

Ce projet constitue une base solide pour des évolutions futures vers une solution plus complète, notamment vers une architecture web avec base de données centralisée et fonctionnalités collaboratives.